

Week 8 Summary Report

Project: E-Commerce Order Analytics System (Intern Mini Project)

Step 1: Realistic Dataset Generation

Generated 4 raw CSV datasets (customers, products, orders, order_items) with intentional data-quality issues (nulls, bad dates, negative quantities, invalid emails) using Python.

```
Data generation complete! Exported raw CSVs to 'data\raw':
- customers.csv: 600 rows
- products.csv: 500 rows
- orders.csv: 800 rows
- order_items.csv: 2010 rows
```

Output: raw CSV generation

Step 2: Data Cleaning with Pandas

Cleaned all raw datasets (fixed date formats, removed duplicates/orphans/invalid values), validated emails and referential integrity, and exported cleaned CSVs along with a detailed cleaning report.

```
=====
                        DATA CLEANING PIPELINE REPORT
=====
Timestamp: 2026-07-09 10:43:59

1. Duplicates Removed:
  - customers: 0 duplicates
  - products: 0 duplicates
  - orders: 0 duplicates
  - order_items: 0 duplicates

2. Orders NULL Customer IDs Removed: 43
3. Orders Date Formats Fixed: 54
4. Orders Future Date Removed: 0
5. Order Items Orphans Removed: 124
6. Order Items Invalid Discounts Removed: 0
7. Order Items Zero Quantities Removed: 0

8. Invalid Emails Found (Count: 11):
  - Customer ID: CUST00019 | Name: Diana Roy | Email: diana.roy43example.com
  - Customer ID: CUST00027 | Name: James Davis | Email: james.davis90@gmail.com
  - Customer ID: CUST00035 | Name: Diana Martin | Email: diana.martin59@hotmail.com
  - Customer ID: CUST00085 | Name: Alice Lee | Email: alice.lee79@
  - Customer ID: CUST00151 | Name: Kevin White | Email: kevin.white97@gmail.com
  - Customer ID: CUST00234 | Name: Olivia Lewis | Email: olivia.lewis75@
  - Customer ID: CUST00310 | Name: Ian Davis | Email: ian.davis59@yahoo.com
  - Customer ID: CUST00352 | Name: Sarah Smith | Email: sarah.smith80outlook.com
  - Customer ID: CUST00442 | Name: Diana Brown | Email: diana.brown27@
  - Customer ID: CUST00491 | Name: Ian Roy | Email: ian.roy36@
  - Customer ID: CUST00500 | Name: Jane Thomas | Email: jane.thomas21@hotmail.com

Summary of Exported Clean Datasets:
  - customers_clean.csv: 600 rows
  - products_clean.csv: 500 rows
  - orders_clean.csv: 757 rows
  - order_items_clean.csv: 1886 rows
=====
```

Output: data cleaning pipeline report

```

PS C:\Users\Manoj Pareek\OneDrive\Desktop\bcccc> python scripts/clean_data.py
>>
Data cleaning pipeline execution completed successfully!
Cleaned datasets exported to 'data\cleaned' directory.
Cleaning report saved to 'data\cleaned\cleaning_report.txt'.
PS C:\Users\Manoj Pareek\OneDrive\Desktop\bcccc> python scripts/test_edge_cases.py
Test passed: Future order dates correctly identified and filtered out.
.Test passed: Invalid discount percents (> 100 or < 0) correctly filtered out.
.Test passed: Orphaned order_items correctly identified and removed.
.Test passed: Zero quantity order items correctly filtered out.
.
-----
Ran 4 tests in 0.026s

OK
PS C:\Users\Manoj Pareek\OneDrive\Desktop\bcccc> python scripts/db_setup.py
Connecting to database at 'ecommerce.db'...
Initializing schema from 'sql\schema.sql'...
Schema initialized successfully.
Loading 'customers_clean.csv' into table 'customers'...
Loading 'products_clean.csv' into table 'products'...
Loading 'orders_clean.csv' into table 'orders'...
Loading 'order_items_clean.csv' into table 'order_items'...

Verifying loaded row counts:
- Table 'customers': 600 rows loaded
- Table 'products': 500 rows loaded
- Table 'orders': 757 rows loaded
- Table 'order_items': 1886 rows loaded

```

Output: clean_data.py run, edge-case tests, and database load log

Step 3: Load Cleaned Data into SQL Database

Created the SQLite schema with proper constraints and loaded all cleaned datasets into the database, verifying row counts across all four tables (600 customers, 500 products, 757 orders, 1886 order items).

Step 4: SQL Analytics - Joins & Aggregations

Wrote and executed basic and intermediate SQL queries covering revenue per category, top customers, monthly order counts, delivery-status checks, return-heavy products, and category-wise return rates.

```
=====
Executing queries from: sql\aggregations.sql
=====
```

Query #1:

-- SQL Analysis: Basic & Intermediate Queries

Columns: ['category', 'total_revenue']

Returned 4 rows. First 3 rows:

('Electronics', 588547.13)

('Books', 585884.61)

('Home', 563357.13)

Query #2:

-- 2. Top 10 customers by total order value (net spend)

Columns: ['customer_id', 'customer_name', 'customer_type', 'total_spend', 'total_orders']

Returned 10 rows. First 3 rows:

('CUST00450', 'Laura Hall', 'VIP', 22764.31, 5)

('CUST00538', 'Emma Miller', 'REGULAR', 22250.14, 7)

('CUST00193', 'Julia Garcia', 'VIP', 17239.47, 4)

Query #3:

-- 3. Month-wise order count for the last 12 months

-- Dynamically calculates relative to the latest order in the database

Columns: ['order_month', 'order_count']

Returned 12 rows. First 3 rows:

('2026-06', 102)

('2026-05', 119)

('2026-04', 84)

Query #4:

-- =====

-- INTERMEDIATE QUERIES

-- =====

Columns: ['customer_id', 'customer_name', 'total_orders_placed']

Returned 299 rows. First 3 rows:

('CUST00011', 'Michael Martin', 4)

('CUST00102', 'Laura Lewis', 4)

('CUST00108', 'Hannah Lee', 4)

Output: aggregation queries (Q1-Q4)

```

Query #5:
-- 5. Products that were ordered but had more returns than purchases
-- Returns have quantity < 0, Purchases have quantity > 0
-- Total quantity sum being less than 0 means net returns exceed purchases
Columns: ['product_id', 'product_name', 'category', 'units_purchased', 'units_returned', 'net_quantity']
Returned 3 rows. First 3 rows:
('PROD00115', 'Table 727', 'Home', 2, 4, -2)
('PROD00136', 'Sweater 357', 'Clothing', 4, 7, -3)
('PROD00182', 'Socks 740', 'Clothing', 5, 8, -3)

Query #6:
-- 6. Calculate the return rate (returned items / total items purchased) per category
-- Return Rate = sum of units returned / sum of units purchased
Columns: ['category', 'units_purchased', 'units_returned', 'return_rate_percent']
Returned 4 rows. First 3 rows:
('Books', 1891, 83, 4.39)
('Clothing', 2088, 81, 3.88)
('Electronics', 2055, 60, 2.92)

=====
Executing queries from: sql\window_functions.sql
=====

Query #1:
-- SQL Analysis: Window Functions & CTEs
Columns: ['region_code', 'order_date', 'daily_revenue', 'running_total']
Returned 621 rows. First 3 rows:
('APAC-EAST', '2025-02-04', 977.61, 977.61)
('APAC-EAST', '2025-03-14', 5960.82, 6938.43)
('APAC-EAST', '2025-03-18', 114.52, 7052.95)

```

Output: return analysis queries (Q5-Q6) and running-total window query

Step 5: SQL Analytics - Window Functions & CTEs

Implemented advanced SQL analytics using window functions and CTEs: DENSE_RANK product ranking, LAG/LEAD order-gap analysis, multi-level CTEs, NTILE customer segmentation, YoY comparison, and first/last value category-shift analysis.

```

Query #2:
-- =====
-- 8. Ranking with DENSE_RANK
-- For each category, rank products by total revenue.
-- Products with same revenue should have same rank.
-- =====
Columns: ['category', 'product_name', 'total_revenue', 'rank_in_category']
Returned 486 rows. First 3 rows:
('Books', 'History 936', 20898.4, 1)
('Books', 'Sci-Fi 422', 17594.21, 2)
('Books', 'Fiction 206', 15147.78, 3)

Query #3:
-- =====
-- 9. LAG/LEAD Analysis
-- Calculate days between consecutive orders for each customer.
-- Flag customers with an average gap > 30 days as "At Risk".
-- =====
Columns: ['customer_id', 'order_date', 'previous_order_date', 'days_gap', 'avg_gap', 'status_flag']
Returned 757 rows. First 3 rows:
('CUST00001', '2026-03-29 15:26:31', None, None, 68.7, 'At Risk')
('CUST00001', '2026-06-06 08:08:18', '2026-03-29 15:26:31', 68.7, 68.7, 'At Risk')
('CUST00002', '2026-01-07 05:59:36', None, None, None, 'Active')

Query #4:
-- =====
-- 10. CTE with Multiple Levels
-- Find monthly revenue per customer, categorize them, and count.
-- =====
Columns: ['order_month', 'spend_category', 'customer_count', 'total_revenue']
Returned 39 rows. First 3 rows:
('2026-06', 'High', 3, 36943.95)
('2026-06', 'Low', 64, 128616.42)
('2026-06', 'Medium', 22, 146879.98)

```

Output: DENSE_RANK ranking, LAG/LEAD gap analysis, multi-level CTE

```

Query #5:
-- =====
-- 11. NTILE for Segmentation
-- Divide customers into 4 quartiles based on total lifetime value.
-- =====
Columns: ['customer_id', 'total_value', 'quartile', 'quartile_label']
Returned 427 rows. First 3 rows:
('CUST00450', 22764.31, 1, 'Platinum')
('CUST00538', 22250.14, 1, 'Platinum')
('CUST00193', 17239.47, 1, 'Platinum')

Query #6:
-- =====
-- 12. Year-over-Year Comparison
-- Compare each month's revenue with same month previous year.
-- =====
Columns: ['year', 'month', 'revenue', 'prev_year_revenue', 'yoy_growth_percent']
Returned 18 rows. First 3 rows:
(2026, 6, 312440.34, 49326.84, '533.41%')
(2026, 5, 349768.84, 24615.36, '1320.94%')
(2026, 4, 220532.71, 33702.31, '554.35%')

Query #7:
-- =====
-- 13. First/Last Value Analysis
-- First vs most recent category purchased.
-- =====
Columns: ['customer_id', 'customer_name', 'first_category', 'last_category', 'category_shift']
Returned 427 rows. First 3 rows:
('CUST00001', 'Bob Smith', 'Home', 'Clothing', 'Yes')
('CUST00002', 'Emma Johnson', 'Electronics', 'Home', 'Yes')
('CUST00003', 'David Robinson', 'Clothing', 'Clothing', 'No')

```

Output: NTILE segmentation, YoY comparison, first/last value analysis

```

Query #8:
-- =====
-- 14. Cumulative Distribution
-- Find percentage of total revenue coming from top N% of customers.
-- =====
Columns: ['customer_id', 'revenue', 'cumulative_revenue', 'cumulative_percent']
Returned 427 rows. First 3 rows:
('CUST00450', 22764.31, 22764.31, 1.02)
('CUST00538', 22250.14, 45014.45, 2.01)
('CUST00193', 17239.47, 62253.92, 2.78)

Query #9:
-- =====
-- 15. Self-Join: Products frequently bought together
-- =====
Returned 427 rows. First 3 rows:
('CUST00450', 22764.31, 22764.31, 1.02)
('CUST00538', 22250.14, 45014.45, 2.01)
('CUST00193', 17239.47, 62253.92, 2.78)

Query #9:
-- =====
-- 15. Self-Join: Products frequently bought together
-- =====
('CUST00538', 22250.14, 45014.45, 2.01)
('CUST00193', 17239.47, 62253.92, 2.78)

Query #9:
-- =====
-- 15. Self-Join: Products frequently bought together
-- =====

Query #9:
-- =====
-- 15. Self-Join: Products frequently bought together
-- =====
-- 15. Self-Join: Products frequently bought together
-- =====
Columns: ['product_id_a', 'product_a', 'product_id_b', 'product_b', 'times_bought_together']
Returned 20 rows. First 3 rows:

```

Output: cumulative distribution and self-join (products bought together)

```

Query #9:
-- =====
-- 15. Self-Join: Products frequently bought together
-- =====
-- 15. Self-Join: Products frequently bought together
-- =====
Columns: ['product_id_a', 'product_a', 'product_id_b', 'product_b', 'times_bought_together']
Returned 20 rows. First 3 rows:
('PROD00010', 'Sneakers 656', 'PROD00466', 'Rug 168', 2)
('PROD00016', 'Blender 922', 'PROD00036', 'Headphones 200', 2)
('PROD00018', 'Laptop 891', 'PROD00137', 'Laptop 270', 2)

```

Output: self-join products frequently bought together

Step 6: Cohort & Retention Analysis

Built a cohort-based retention query grouping customers by registration month and calculating month 0-3 active users and retention percentages per cohort.

```

=====
Executing queries from: sql\cohort_analysis.sql
=====

Query #1:
-- SQL Analysis: Cohort & Retention Analysis
Columns: ['cohort_month', 'cohort_size', 'month_0_active_users', 'month_1_active_users', 'month_2_active_users', 'month_3_active_users',
mont ('PROD00018', 'Laptop 891', 'PROD00137', 'Laptop 270', 2)

=====
Executing queries from: sql\cohort_analysis.sql
=====

Query #1:
-- SQL Analysis: Cohort & Retention Analysis
Columns: ['cohort_month', 'cohort_size', 'month_0_active_users', 'month_1_active_users', 'month_2_active_users', 'month_3_active_users',
mont

Query #1:
-- SQL Analysis: Cohort & Retention Analysis
Columns: ['cohort_month', 'cohort_size', 'month_0_active_users', 'month_1_active_users', 'month_2_active_users', 'month_3_active_users',
month_0_retention_pct', 'month_1_retention_pct', 'month_2_retention_pct', 'month_3_retention_pct']
Returned 17 rows. First 3 rows:
('2025-01', 34, 4, 1, 4, 1, 11.76, 2.94, 11.76, 2.94)
('2025-02', 36, 0, 3, 3, 1, 0.0, 8.33, 8.33, 2.78)
('2025-03', 40, 1, 6, 3, 5, 2.5, 15.0, 7.5, 12.5)

Returned 17 rows. First 3 rows:
('2025-01', 34, 4, 1, 4, 1, 11.76, 2.94, 11.76, 2.94)
('2025-02', 36, 0, 3, 3, 1, 0.0, 8.33, 8.33, 2.78)
('2025-03', 40, 1, 6, 3, 5, 2.5, 15.0, 7.5, 12.5)

('2025-02', 36, 0, 3, 3, 1, 0.0, 8.33, 8.33, 2.78)
('2025-03', 40, 1, 6, 3, 5, 2.5, 15.0, 7.5, 12.5)

```

Output: cohort & retention analysis query results

Step 7: CLI Reporting Tool

Developed a command-line reporting tool that takes report type and date range as input, connects to SQLite, and generates monthly/daily summary reports with key metrics, top products, and period-over-period % change.

```
=====
E-COMMERCE ORDER ANALYTICS SUMMARY REPORT
Report Type: MONTHLY
Current Period: 2026-06-01 to 2026-06-30
Previous Period: 2026-05-02 to 2026-05-31
=====

[KEY PERFORMANCE METRICS]
-----+-----+-----+-----
Metric      | Current Period | Previous Period | % Change
-----+-----+-----+-----
Total Orders | 102            | 116             | -12.07%
Net Revenue  | 312,440.34     | 346,159.19     | -9.74%
Unique Customers | 89            | 100             | -11.00%
-----+-----+-----+-----

[TOP 3 PRODUCTS BY REVENUE]
-----+-----+-----+-----
Product Name | Category      | Units Sold | Revenue
-----+-----+-----+-----
Sneakers 455 | Clothing      | 21         | 10,671.99
Smartwatch 908 | Electronics   | 15         | 7,159.77
Cookware 961  | Home          | 22         | 6,975.32
-----+-----+-----+-----

[PERIODIC BREAKDOWN (MONTHLY)]
-----+-----+-----+-----
Period | Orders | Revenue | Unique Customers
-----+-----+-----+-----
2026-06 | 102    | 312,440.34 | 89
-----+-----+-----+-----
```

Output: monthly summary report from CLI tool

```
|
=====
E-COMMERCE ORDER ANALYTICS SUMMARY REPORT
Report Type: DAILY
Current Period: 2026-06-01 to 2026-06-05
Previous Period: 2026-05-27 to 2026-05-31
=====

[KEY PERFORMANCE METRICS]
-----+-----+-----+-----
Metric      | Current Period | Previous Period | % Change
-----+-----+-----+-----
Total Orders | 21             | 27             | -22.22%
Net Revenue  | 68,851.22      | 84,994.10     | -18.99%
Unique Customers | 20            | 26            | -23.08%
-----+-----+-----+-----

[TOP 3 PRODUCTS BY REVENUE]
-----+-----+-----+-----
Product Name | Category | Units Sold | Revenue
-----+-----+-----+-----
Sneakers 455 | Clothing | 13         | 6,614.95
Sneakers 888 | Clothing | 8          | 4,789.90
Rug 864      | Home    | 7          | 4,491.55
-----+-----+-----+-----

[PERIODIC BREAKDOWN (DAILY)]
-----+-----+-----+-----
Period      | Orders | Revenue | Unique Customers
-----+-----+-----+-----
2026-06-01 | 6      | 20,515.93 | 6
2026-06-02 | 4      | 14,901.69 | 4
2026-06-03 | 5      | 12,734.94 | 5
2026-06-04 | 3      | 12,593.76 | 3
2026-06-05 | 3      | 8,104.89  | 3
-----+-----+-----+-----
```

Output: daily summary report from CLI tool

Step 8: Edge Case Handling

Wrote and passed automated test cases covering future order dates, invalid discount percentages, orphaned order items, and zero-quantity order items (see edge-case test results above under Step 2).